

```

import json
import os
from datetime import datetime

# — Constants —————
DATA_FILE = "expenses.json"
SENTINEL = "done" # Kill switch – graceful shutdown

CATEGORIES = [
    "Food", "Transport", "Shopping",
    "Bills", "Health", "Education", "Other"
]

# — Persistence Helpers (RAM → Disk via JSON Serialization) —

def load_expenses():
    """Load saved expense records from disk into memory."""
    if os.path.exists(DATA_FILE):
        with open(DATA_FILE, "r") as f:
            return json.load(f)
    return [] # Fresh start if no file exists

def save_expenses(expenses):
    """Serialize the in-memory list to disk so data survives restart."""
    with open(DATA_FILE, "w") as f:
        json.dump(expenses, f, indent=4)

def get_next_id(expenses):
    """Generate a unique ID that safely accounts for deleted items."""
    return max([exp["id"] for exp in expenses], default=0) + 1

# — Phase 1: INPUT – The Gatekeeper —————

def get_valid_amount():
    while True:
        raw = input(" 💰 Enter amount (or type 'done' to finish) : ").strip()

        if raw.lower() == SENTINEL:
            return SENTINEL

        try:
            amount = float(raw)
            if amount <= 0:
                print(" ⚠ Amount must be greater than 0. Try again.")
                continue
            return amount
        except ValueError:
            print(" ⚠ Invalid input. Please enter a number (e.g., 100, 50.5).")

def get_category():
    print("\n 📁 Categories:")
    for i, cat in enumerate(CATEGORIES, start=1):
        print(f" {i}. {cat}")
    while True:
        try:
            choice = int(input(" Choose category (1-7) : "))
            if 1 <= choice <= len(CATEGORIES):
                return CATEGORIES[choice - 1]
            print(" ⚠ Enter a number between 1 and 7.")
        except ValueError:
            print(" ⚠ Please enter a valid number.")

```

```
def get_description():
    desc = input(" 📝 Description (e.g., Lunch, Bus ticket) : ").strip()
    return desc if desc else "No description"
```

— Phase 2: PROCESS – The Accumulator Engine —————

```
def add_expense(expenses):
    print("\n" + "-" * 50)
    print("  +  ADD NEW EXPENSE")
    print("-" * 50)

    amount = get_valid_amount()
    if amount == SENTINEL:
        return False

    category = get_category()
    description = get_description()

    record = {
        "id" : get_next_id(expenses),
        "amount" : round(amount, 2),
        "category" : category,
        "description": description,
        "date" : datetime.now().strftime("%Y-%m-%d %H:%M")
    }

    expenses.append(record)
    save_expenses(expenses)

    print(f"\n  ✅ Expense of ₹{amount:.2f} added under '{category}'!")
    return True
```

— Phase 3: OUTPUT – The Display / View Layer —————

```
def view_expenses(expenses):
    print("\n" + "=" * 55)
    print("  📑 EXPENSE LEDGER")
    print("=" * 55)

    if not expenses:
        print("  🚫 No expenses recorded yet.")
        print("=" * 55)
        return

    for exp in expenses:
        print(f"  [{exp['id']:02d}] {exp['date']} | {exp['category']:<12}"
              f" | ₹{exp['amount']:>8.2f} | {exp['description']}")

    print("-" * 55)
    display_summary(expenses)
    print("=" * 55)

def display_summary(expenses):
    total = 0.0
    highest = 0.0
    highest_item = ""
    category_totals = {}

    for exp in expenses:
        amount = exp["amount"]
        total += amount

        if amount > highest:
```

```

        highest      = amount
        highest_item = exp["description"]

    cat = exp["category"]
    category_totals[cat] = category_totals.get(cat, 0) + amount

average = total / len(expenses) if expenses else 0

print(f"\n 💰 TOTAL SPENT      : ₹{total:.2f}")
print(f" 📊 AVERAGE/EXPENSE : ₹{average:.2f}")
print(f" ⬆️ HIGHEST EXPENSE : ₹{highest:.2f} ({highest_item})")
print(f"\n 📁 CATEGORY BREAKDOWN:")
for cat, amt in sorted(category_totals.items(), key=lambda x: x[1], reverse=True):
    bar = "█" * int((amt / total) * 20) if total > 0 else ""
    print(f"      {cat:<12} ₹{amt:>8.2f} {bar}")

def delete_expense(expenses):
    view_expenses(expenses)
    if not expenses:
        return

    try:
        choice = int(input("\nEnter expense ID to delete : "))
        for i, exp in enumerate(expenses):
            if exp["id"] == choice:
                removed = expenses.pop(i)
                save_expenses(expenses)
                print(f"\n 🗑 Deleted: ₹{removed['amount']:.2f} - {removed['description']}")
                return
        print(" ⚠ ID not found.")
    except ValueError:
        print(" ⚠ Please enter a valid number.")

def quick_add_session(expenses):
    print("\n" + "=" * 50)
    print(" ⚡ QUICK-ADD SESSION (type 'done' to stop)")
    print("=" * 50)

    session_total = 0.0
    count = 0

    while True:
        print(f"\n Running Total: ₹{session_total:.2f} | Entries: {count}")
        print("-" * 50)

        amount = get_valid_amount()
        if amount == SENTINEL:
            break

        category = get_category()
        description = get_description()

        record = {
            "id"      : get_next_id(expenses),
            "amount"   : round(amount, 2),
            "category" : category,
            "description": description,
            "date"     : datetime.now().strftime("%Y-%m-%d %H:%M")
        }

        expenses.append(record)
        session_total += amount
        count += 1

    print(f" ✅ Added ₹{amount:.2f}. Session total: ₹{session_total:.2f}")

```

```

save_expenses(expenses)

print("\n" + "=" * 50)
print(f" 🏁 SESSION COMPLETE")
print(f" 📦 Expenses Added : {count}")
print(f" 💰 SESSION TOTAL : ₹{session_total:.2f}")
print("=" * 50)

```

— Main Menu —————

```

def show_menu():
    print("\n" + "=" * 50)
    print(" 📁 DecodeLabs – Expense Tracker")
    print("=" * 50)
    print(" 1. ➕ Add Single Expense")
    print(" 2. ⚡ Quick-Add Session (Continuous Mode)")
    print(" 3. 📋 View All Expenses & Summary")
    print(" 4. 🗑 Delete an Expense")
    print(" 5. 🚪 Exit & Show Final Total")
    print("-" * 50)

def main():
    print("\n Welcome to DecodeLabs Expense Tracker! 📁\n")

    expenses = load_expenses()
    print(f" 📁 Loaded {len(expenses)} existing expense(s) from storage.")

    while True:
        show_menu()
        choice = input(" Choose an option (1-5) : ").strip()

        if choice == "1":
            if not add_expense(expenses):
                break
        elif choice == "2":
            quick_add_session(expenses)
        elif choice == "3":
            view_expenses(expenses)
        elif choice == "4":
            delete_expense(expenses)
        elif choice == "5":
            print("\n" + "=" * 50)
            print(" 🏁 FINAL REPORT")
            print("=" * 50)
            view_expenses(expenses)
            print("\n 🙌 Goodbye! Keep tracking. – DecodeLabs\n")
            break
        else:
            print("\n ⚠ Invalid choice. Please enter 1 to 5.")

if __name__ == "__main__":
    main()

```